



university of
 groningen

faculty of science
and engineering

Towards Efficient Galerkin Projection in Lagrangian ROMs for Sloshing

Internship MSc Applied Mathematics

January 2026

Student: M.A. Mandeville

Company/Institute: The von Karman Institute for Fluid Dynamics

External supervisors: prof. M.A. Mendez, ir. S. Ahizi

Internal RUG first assessor: dr. J. Koellermeier

Internal RUG second assessor: dr. ir. R. Luppés

Abstract

This work investigates projection-based Reduced Order Models (ROMs) for Lagrangian SPH sloshing flows. We introduce a global algebraic reformulation of the SPH equations, replacing standard neighbour loops with dense matrix-vector operations to enable both rigorous operator projection and efficient GPU-based integration for moderate particle counts. Using Proper Orthogonal Decomposition (POD) and Galerkin projection, we aim to recover a canonical Linear Time-Invariant (LTI) system. However, our analysis demonstrates that the nonlinear kernel transformations inherent to particle interactions introduce a factorized dynamics structure, where the linear system is explicitly modulated by state-dependent terms. To this end, we evaluate three strategies to approximate this nonlinearity highlighting the limitations of linearisation and Random Fourier Features for compact support kernels. Consequently, we propose an SVD-based interpolation framework as the designated strategy for future work.

Keywords: Lagrangian framework, Smoothed-Particle Hydrodynamics, reduced Order Model, Proper Orthogonal Decomposition, Singular Value Decomposition

Contents

1	Introduction	3
2	SPH equations	4
2.1	Smoothing kernels	4
2.2	Discretisation of the governing equations	6
2.3	Matrix form	9
3	Reduced Order Model	13
3.1	SVD/POD	13
3.2	Galerkin projection	16
3.3	Boundary conditions	16
4	Radial derivative	17
4.1	Linearisation	17
4.2	Random Fourier Features	19
4.3	SVD Interpolation	20
5	Conclusion and discussion	22
6	References	23

1 Introduction

The von Karman Institute for Fluid Dynamics (VKI) is an educational and scientific non-profit organisation located in Sint-Genesius-Rode, Belgium. VKI focuses on three specific fields: aeronautics and aerospace, environment and applied fluid dynamics, turbomachinery and propulsion.

This project was done via the Short Training Program (STP) offered by the institute, and it fits within both the aerospace and aeronautics and the environmental and applied fluid dynamics departments. The former department focuses on modelling, simulating, and experimental validation of atmospheric entry flows and thermal protection systems. The latter, on the other hand, focuses on all natural and industrial processes in which fluid dynamics play an important role.

The Propellant Management Research Expertise Group (REG) at VKI was created in the early 2000s to support space propulsion needs. Since then, the group conducts advanced research on cryogenic fluid dynamics, with applications in space propulsion, aeronautics, maritime transport and energy systems. In particular, fundamental research is undertaken on phase change, two-phase flows, thermal stratifications, and sloshing, using specialised facilities. These facilities include e.g. nitrogen facilities and sloshing tables.

Cryogenic propellants – such as liquid hydrogen or liquid oxygen – are used for instance in spatial applications for their high energy density. Critical research on propellant management is carried out for safe and cost-effective deep-space missions. During various manoeuvres and docking operations, the sloshing of the propellant leads to thermal mixing, de-stratification, and localised phase transitions phenomena, which are not fully resolved under current CFD and reduced order models [16].

To improve existing simulations, there are three main research points, which include reduced order models capable of predicting sloshing and two-phase instabilities in real time.

Typically, fluid flows are modelled using Finite Volumes in a Eulerian framework. That is, we discretise the domain into a grid and evaluate the flow at either the cell centres or cell faces. However, when considering free-surface flows, e.g. sloshing, we have the problem of determining exactly where the interface between the fluid and the gas is. Hence, for such flows, it can be advantageous to instead discretise the fluid itself. In other words, we replace the continuous fluid by a discrete set of particles, with a set of forces between them to replicate the pressure and other forces acting on the fluid. We consider here one such example of Lagrangian meshfree method: Smoothed Particle Hydrodynamics (SPH).

SPH was developed independently by Lucy [12] and Gingold and Monaghan [8] in 1977, originally for astrophysical purposes. Since then, and especially in the last two decades, the scope of applications has been greatly expanded. Namely, SPH is now commonly used for free-surface flows, of which sloshing is the application of interest here.

There are two underlying steps to this method. Firstly, a function is interpolated by convolution with a smoothing kernel. Secondly, this continuous convolution is discretised as a sum over the particles making up the fluid. In real-life scenarios – especially in the case of three-dimensional flows – the number of particles can be extremely large. Hence, we aim to obtain a reduced order model to decrease the degrees of freedom of the computations.

Section 2 provides a look into the SPH equations both as sums over neighbouring particles and as matrix-vector operations. Section 3 is dedicated to the derivation of a reduced order model (ROM) by using Singular Value Decomposition (SVD) and Galerkin projection. A nonlinear term is present in this ROM, which is analysed in Section 4. Finally, a conclusion and a discussion is given in Section 5.

2 SPH equations

In this section we look at the governing equations for the smoothed particle hydrodynamics model. This method consists of two main steps. We first apply convolution with a smoothing kernel, and then approximate this continuous convolution by a sum. After looking at the kernel functions, we turn to the equations of momentum and continuity, as well as the equation of state for pressure. This leads us to a system of ODE's given by sums over particles, which we aim to rewrite as standard matrix-vector operations.

2.1 Smoothing kernels

2.1.1 Continuous and discrete interpolation

The first step of smoothed particle hydrodynamics is kernel interpolation [8, 12]. Let $f(\mathbf{x})$, $\mathbf{x} \in \mathbb{R}^{n_d}$ be a smooth function. Then, the integral form of the interpolation is defined by a convolution between the function f and a kernel W :

$$\langle f \rangle(\mathbf{x}) := \int f(\mathbf{x}') W(\mathbf{x} - \mathbf{x}', \varepsilon) d\mathbf{x}', \quad (2.1)$$

where the integral is taken over the whole flow domain and $W(\mathbf{x}, \varepsilon)$ is the smoothing kernel function, which satisfies

$$\int W(\mathbf{x}, \varepsilon) d\mathbf{x} = 1, \quad \lim_{\varepsilon \rightarrow 0} W(\mathbf{x}, \varepsilon) = \delta(\mathbf{x}), \quad (2.2)$$

for δ the Dirac delta function. We call ε the smoothing length, and from the limiting profile, it can be seen that ε determines the spread of the kernel function W . We also assume that W has compact support, that is $W(\mathbf{x}, \varepsilon) = 0$ whenever $\|\mathbf{x}\| \geq k\varepsilon$, for some $k \in \mathbb{R}^+$. In general, ε is close to the initial particle spacing, and the kernel W is zero beyond a distance of 2ε .

Discretising the above continuous convolution (2.1) gives the discrete SPH approximation as:

$$f(\mathbf{x}_i) := \sum_j m_j \frac{f(\mathbf{x}_j)}{\rho_j} W_{ij}, \quad (2.3)$$

where $i, j \in \{1, \dots, n_p\}$ for n_p the number of particles considered, and m_j/ρ_j is the volume element of particle j consisting of the mass and pressure respectively. We make use of the shorthand notation

$$W_{ij} = W(\mathbf{x}_i - \mathbf{x}_j, \varepsilon), \quad (2.4)$$

for ε typically being a known constant value.

Hence, the value of f at particle i is defined as a sum over all other particles. This is where the assumption of compact support is crucial. We do not aim to sum over *all* particles, since particles that are far away from particle i will not influence it. So, in reality, we aim to sum only over the neighbours of the particle.

2.1.2 Gradient of the kernel

We can also consider interpolation with the gradient of f , namely:

$$\langle \nabla f \rangle(\mathbf{x}) = \int f(\mathbf{x}') \nabla W(\mathbf{x} - \mathbf{x}', \varepsilon) d\mathbf{x}', \quad (2.5)$$

where we make use of the following property of convolution:

$$\frac{\partial f}{\partial x} * W = f * \frac{\partial W}{\partial x}. \quad (2.6)$$

The discrete gradient of f is thus obtained by discretising the above integral:

$$\nabla f(\mathbf{x}_i) = \sum_j m_j \frac{f(\mathbf{x}_j)}{\rho_j} \nabla W_{ij}, \quad (2.7)$$

where the gradient on the right hand side is taken with respect to particle i .

Typically, kernels are symmetric functions, where to ensure this symmetry we have that W is a function of the radial component $r = \|\mathbf{x}\|$ and the smoothing length ε . Hence, by the chain rule:

$$\nabla W = \frac{\mathbf{x}}{r} \frac{\partial W}{\partial r} + \frac{\partial W}{\partial \varepsilon} \nabla \varepsilon. \quad (2.8)$$

The smoothing length is generally taken to be constant, so the second term on the right hand side vanishes. Denoting the radial derivative by

$$S := \frac{1}{r} \frac{\partial W}{\partial r}, \quad (2.9)$$

the gradient of the kernel simply becomes $\nabla W = \mathbf{x}S$. Therefore:

$$\nabla W_{ij} = (\mathbf{x}_i - \mathbf{x}_j) S_{ij} =: \underline{\mathbf{X}}_{ij}^d S_{ij}, \quad (2.10)$$

where the radial derivative S_{ij} is taken with respect to the i -th particle, and we make use of the shorthand notation $\underline{\mathbf{X}}^d$ to denote the tensor consisting of differences in position. Its ij -th entry is then a vector consisting of the difference in position between particle i and j . All throughout this paper, the superscript d will be used to denote such a difference.

There are many examples of smoothing kernels, as given by Voileau [20] and Liu and Liu [10]. Two are highlighted here, the Gaussian and Wendland kernels, which are used in Section 4.

2.1.3 Gaussian kernel

Despite not having compact support, a Gaussian does approach zero rapidly enough to be considered a suitable kernel function. This kernel is given by:

$$W(r, \varepsilon) = \alpha_d \exp\left(-\frac{r^2}{\varepsilon^2}\right), \quad (2.11)$$

where α_d is a normalisation constant which depends on the number n_d of spatial dimensions considered. Taking the radial derivative (2.9) yields:

$$S = \frac{1}{r} \frac{\partial W}{\partial r} = \frac{2\alpha_d}{\varepsilon^2} \exp\left(-\frac{r^2}{\varepsilon^2}\right) = \frac{2}{\varepsilon^2} W(r, \varepsilon). \quad (2.12)$$

2.1.4 Wendland kernel

The Wendland kernel [21] is defined on $0 \leq r \leq 2\varepsilon$ by

$$W(r, \varepsilon) = \alpha_d \left(1 - \frac{r}{2\varepsilon}\right)^4 \left(\frac{2r}{\varepsilon} + 1\right), \quad 0 \leq \frac{r}{\varepsilon} \leq 2, \quad (2.13)$$

and is set to zero otherwise. Taking the radial derivative gives:

$$S = \frac{1}{r} \frac{\partial W}{\partial r} = \frac{-5\alpha_d}{\varepsilon} \left(1 - \frac{r}{2\varepsilon}\right)^3. \quad (2.14)$$

From the above examples of radial derivatives, we can see that S is defined on $r \in [0, \infty)$, and in fact, for most kernels, it is generally well defined at $r = 0$.

2.2 Discretisation of the governing equations

Using kernel interpolation, and particularly the discrete approximations, we can write the governing equations for particle i as sums over its neighbouring particles.

2.2.1 Momentum equation

The governing equations for fluid flow are given by the Navier-Stokes equations. In Lagrangian form, the conservation of momentum can be written as:

$$\frac{d\mathbf{u}}{dt} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{a}_{\text{ext}}, \quad (2.15)$$

where on the left hand side we have the material or total derivative of the velocity $\mathbf{u} \in \mathbb{R}^{n_d}$, p is the pressure, ρ the density and ν the kinematic viscosity. We have three force terms on the right hand side, the pressure force $(-\nabla p/\rho)$, the viscous force $(\nu \nabla^2 \mathbf{u})$ and the external forces $(\mathbf{a}_{\text{ext}})$ representing the external acceleration (e.g. gravitational acceleration) and the boundary force. Denoting the sum

of these forces by $\mathbf{f}$, we obtain the following equation for the acceleration of particle i :

$$\frac{d\mathbf{u}_i}{dt} = \mathbf{f}_i, \quad (2.16)$$

where $\mathbf{u}_i \in \mathbb{R}^{n_d}$ is the velocity of particle i and $\mathbf{f}_i \in \mathbb{R}^{n_d}$ is the sum of the pressure, viscous and external forces evaluated at the location of particle i . Applying kernel interpolation first on the pressure force yields, using the discrete gradient (2.7):

$$\mathbf{f}_i^p := -\frac{1}{\rho} \nabla p(\mathbf{x}_i) = -\sum_j m_j \frac{p_j}{\rho_j} \nabla W_{ij}, \quad (2.17)$$

where m_j, p_j, ρ_j are the mass, pressure and density of particle j respectively. One can observe that (2.17) is not symmetric. Indeed, when considering the case where only two particles interact, the influence of particle i on particle j is not the same as that of j on i . This antisymmetry breaks the conservation of momentum. To counter this, different symmetrisations for (2.17) have been suggested [13, 14], we use the following [6]:

$$\mathbf{f}_i^p := -\sum_j m_j \frac{p_i + p_j}{\rho_i \rho_j} \nabla W_{ij}. \quad (2.18)$$

The derivation of the viscous force is similar to that of the pressure force, with the Laplacian of the kernel. Rewriting the Laplacian in terms of the gradient, and symmetrising, yields [3, 6, 11]:

$$\mathbf{f}_i^\nu := 4\nu \sum_j m_j \frac{(\mathbf{x}_i - \mathbf{x}_j) \cdot \nabla W_{ij}}{(\rho_i + \rho_j) \|\mathbf{x}_i - \mathbf{x}_j\|^2} (\mathbf{u}_i - \mathbf{u}_j). \quad (2.19)$$

The discrete equation of momentum is then given by

$$\frac{d\mathbf{u}_i}{dt} = \mathbf{f}_i^p + \mathbf{f}_i^\nu + \mathbf{a}_{\text{ext},i}, \quad (2.20)$$

where the first two terms on the right-hand side are respectively (2.18) and (2.19).

2.2.2 Continuity equation

Historically, in SPH, the density of particle i is computed as a sum over its neighbours [8]:

$$\rho_i = \sum_j m_j W_{ij}. \quad (2.21)$$

This form conserves mass and works well for astrophysical applications. However, when modelling free surface flows, this gives incorrect values for the pressure at the interface, due to the smaller number of neighbouring particles [1, 13]. Instead, we look at the continuity equation in Lagrangian form, which can be written as:

$$\frac{d\rho}{dt} = \mathbf{u} \cdot \nabla \rho - \nabla \cdot (\rho \mathbf{u}), \quad (2.22)$$

with on the left hand side the material or total derivative. Then, applying kernel interpolation and discretising yields the discrete continuity equation:

$$\frac{d\rho_i}{dt} = \sum_j m_j (\mathbf{u}_i - \mathbf{u}_j) \cdot \nabla W_{ij}. \quad (2.23)$$

Each particle initially has a base density ρ_0 , and the relative motion of particles influences the change in density. This does yield correct pressure and density at the free surface [1].

Looking at the form (2.23), one can observe that we are not in a compressible case anymore. Indeed, since we do have a change in density, we are in the case of Weakly Compressible Smoothed Particle Hydrodynamics (WCSPH). This is an important notion that will come useful further on.

2.2.3 Equation of state

There are many equations relating the pressure and the density. One being Tait's equation of state:

$$p = \frac{c_s^2 \rho_0}{\gamma} \left(\left(\frac{\rho}{\rho_0} \right)^\gamma - 1 \right), \quad (2.24)$$

where c_s is the speed of sound, ρ_0 the base density of the fluid, and γ is a parameter, typically either 7 or 1. In our case, we consider $\gamma = 1$, so that the equation of state simplifies to

$$p = c_s^2 (\rho - \rho_0). \quad (2.25)$$

2.2.4 Discrete equations of motion

Summarising the previous results – using the form of the gradient of the kernel as in (2.10) and substituting the pressure from the equation of state (2.25) – yields the following system of ODEs

$$\begin{aligned} \frac{d\mathbf{u}_i}{dt} &= \mathbf{f}_i^p + \mathbf{f}_i^\nu, \\ \mathbf{f}_i^p &= -c_s^2 \sum_j m_j \frac{\rho_i + \rho_j - 2\rho_0}{\rho_i \rho_j} \underline{\mathbf{X}}_{ij}^d S_{ij}, \\ \mathbf{f}_i^\nu &= 4\nu \sum_j m_j \frac{1}{\rho_i + \rho_j} \underline{\mathbf{U}}_{ij}^d S_{ij}, \\ \frac{d\rho_i}{dt} &= \sum_j m_j \underline{\mathbf{X}}_{ij}^d \cdot \underline{\mathbf{U}}_{ij}^d S_{ij}, \end{aligned} \quad (2.26)$$

where $\underline{\mathbf{U}}_{ij}^d = \mathbf{u}_i - \mathbf{u}_j \in \mathbb{R}^{n_d}$ is the difference in velocity – using the same notation as in (2.10).

2.2.5 Boundary conditions

A full analysis on the treatment of the boundary conditions is out of the scope of this paper, however we do mention them for the sake of completeness. The boundary is also discretised, we have a subset

B of boundary particles. There are then two main ways to continue.

Domínguez *et al.* [6] solve the SPH equations (2.26) for these particles, where the acceleration of the boundary particles is set to zero,

$$\left. \frac{d\mathbf{u}}{dt} \right|_{i \in B} = 0, \quad (2.27)$$

for a boundary that is considered static. For a moving boundary, the right hand side of (2.27) is set to a constant force F_{fixed} .

On the other hand, Monaghan [13] considers a repulsive force between the boundary and fluid particles, depending on the distance r between them. This force is of the form

$$f(r) = D \left(\left(\frac{r_0}{r} \right)^{p_1} - \left(\frac{r_0}{r} \right)^{p_2} \right) \frac{\mathbf{r}}{r^2}, \quad (2.28)$$

where $\mathbf{r}$ is the difference in positions between the particles, r_0 is the initial distance between particles, D is a problem specific coefficient and p_1, p_2 are constants which satisfying $p_1 > p_2$. In order for the force to be repulsive, it is set to zero whenever $r > r_0$.

2.3 Matrix form

As seen in (2.26), we have a system of ODEs consisting of sums over particles, and in particular over neighbouring particles. When implementing this in a code, we therefore first need to find the neighbours of particle i . This is done by looping over all particles j and checking if the distance between these two particles i and j is smaller than some pre-determined fixed radius (which depends on ε). The number of particles n_p can be rather large – especially so when considering a three dimensional flow – so these sums and loops can become computationally quite heavy.

So, instead of considering sums, we aim to rewrite the above system of equations as matrix-vector operations. These operations – albeit consisting of large matrices – are then easier to compute than the neighbour search, as certain elements may be pre-computed in advance.

Note, however, that we must be careful when comparing sums and loops to matrix operations. In practice, matrix operations are more computationally efficient on optimized code for a reasonable number of particles. After some threshold, matrices become too large, and memory becomes a constraint. It thus becomes preferable to use sums over nearby particles for very large particle counts. Regardless, there is still a relatively large window where matrix operations simplify the simulations compared to the neighbour search, and that is the range that we work in here.

2.3.1 Separation of spatial dimensions

The first step to rewrite the equations from sums to matrix operations is to separate the spatial dimensions. In the following, we suppose that we have a two dimensional flow, so $n_d = 2$ and

$$\mathbf{x} = (x, y), \quad \mathbf{u} = (u, v). \quad (2.29)$$

The subsequent results also hold for three dimensional flow, with the extra dimension z and corresponding velocity w . The difference in position, denoted by the tensor $\underline{X}^d$, is then split into two matrices, of dimension $n_p \times n_p$ both, and similarly for the difference in velocity:

$$\underline{X}^d = (X^d, Y^d), \quad \underline{U}^d = (U^d, V^d). \quad (2.30)$$

2.3.2 Momentum equation

The acceleration of particle i is also a vector of size n_d , and so are the pressure and viscous forces. Hence, we can also split them into their x - and y -components. We only write the results for the x -component, the same holds for y by simply substituting the corresponding variables. For the viscous force in the x -direction we have, for e_x the appropriate standard basis vector of $\mathbb{R}^{n_d}$,

$$\mathbf{f}_i^\nu \cdot e_x = 4\nu \sum_j m_j \frac{S_{ij}}{\rho_i + \rho_j} U_{ij}^d. \quad (2.31)$$

We aim to derive a vector (obtained by matrix-vector operations) for which the i -th entry is (2.31).

Illustrating in the case for $n_p = 3$, the denominator $\rho_i + \rho_j$ corresponds to the ij -th entry of:

$$\begin{bmatrix} 2\rho_1 & \rho_1 + \rho_2 & \rho_1 + \rho_3 \\ \rho_2 + \rho_1 & 2\rho_2 & \rho_2 + \rho_3 \\ \rho_3 + \rho_1 & \rho_3 + \rho_2 & 2\rho_3 \end{bmatrix} = \begin{bmatrix} \rho_1 \\ \rho_2 \\ \rho_3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} \rho_1 & \rho_2 & \rho_3 \end{bmatrix}. \quad (2.32)$$

Hence, in the general case of n_p particles, the denominator is the ij -th entry of

$$\hat{\boldsymbol{\rho}} := \boldsymbol{\rho} \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} \boldsymbol{\rho}^T \in \mathbb{R}^{n_p \times n_p}, \quad (2.33)$$

where $\boldsymbol{\rho} \in \mathbb{R}^{n_p}$ is a vector of densities and $\mathbf{1}_{n,m} \in \mathbb{R}^{n \times m}$ denotes a matrix of ones. From this, we can rewrite

$$\frac{S_{ij}}{\rho_i + \rho_j} = \frac{S_{ij}}{\hat{\rho}_{ij}}, \quad (2.34)$$

which corresponds to a component-wise division, also known as a *Hadamard division*, denoted $\oslash$. We also note that we have a component-wise multiplication $S_{ij} U_{ij}^d$, which corresponds to a *Hadamard product*, denoted $\odot$. Hence, combining the two, we obtain the following matrix operation:

$$U^d \oslash \hat{\boldsymbol{\rho}} \odot S \in \mathbb{R}^{n_p \times n_p}, \quad (2.35)$$

where we make use of the fact that Hadamard operations commute.

Finally, the sum over particles j and the multiplication by the associated mass is obtained by a matrix-vector multiplication, where $\mathbf{m} \in \mathbb{R}^{n_p}$ is a vector of masses. Therefore

$$\mathbf{F}_x^\nu := 4\nu (U^d \oslash \hat{\boldsymbol{\rho}} \odot S) \mathbf{m} \in \mathbb{R}^{n_p} \quad (2.36)$$

is a vector for which the i -th entry corresponds to (2.31).

We proceed similarly for the pressure force, which in the x -direction corresponds to:

$$\mathbf{f}_i^p \cdot \mathbf{e}_x = -c_s^2 \sum_j m_j \frac{\rho_i + \rho_j - 2\rho_0}{\rho_i \rho_j} X_{ij}^d S_{ij}. \quad (2.37)$$

The numerator $\rho_i + \rho_j - 2\rho_0$ corresponds to the ij -th entry of the linear combination

$$\tilde{\boldsymbol{\rho}} := \boldsymbol{\rho} \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} \boldsymbol{\rho}^T - 2\rho_0 \mathbf{1}_{n_p \times n_p} \in \mathbb{R}^{n_p \times n_p}, \quad (2.38)$$

and the denominator $\rho_i \rho_j$ to the ij -th entry of – illustrating first for $n_p = 3$:

$$\begin{bmatrix} \rho_1^2 & \rho_1 \rho_2 & \rho_1 \rho_3 \\ \rho_2 \rho_1 & \rho_2^2 & \rho_2 \rho_3 \\ \rho_3 \rho_1 & \rho_3 \rho_2 & \rho_3^2 \end{bmatrix} = \begin{bmatrix} \rho_1 \\ \rho_2 \\ \rho_3 \end{bmatrix} \begin{bmatrix} \rho_1 & \rho_2 & \rho_3 \end{bmatrix}, \quad (2.39)$$

and so for n_p particles it corresponds to the ij -th entry of

$$\boldsymbol{\rho} \boldsymbol{\rho}^T. \quad (2.40)$$

Hence, applying Hadamard operation and matrix-vector multiplication, we obtain

$$F_x^p := -c_s^2 (\tilde{\boldsymbol{\rho}} \odot X^d \odot S \odot (\boldsymbol{\rho} \boldsymbol{\rho}^T)) \mathbf{m} \in \mathbb{R}^{n_p}, \quad (2.41)$$

for which the i -th entry corresponds to (2.37).

Thus, summing the two forces and factoring the radial derivative S yields the acceleration of particle i in the x -direction as

$$\frac{d\mathbf{u}}{dt} = [(-c_s^2 X^d \odot \tilde{\boldsymbol{\rho}} \odot (\boldsymbol{\rho} \boldsymbol{\rho}^T) + 4\nu U^d \odot \hat{\boldsymbol{\rho}}) \odot S] \mathbf{m} + \mathbf{a}_{\text{ext},x}, \quad (2.42)$$

where $\mathbf{u} \in \mathbb{R}^{n_p}$ is a vector of the x -components of particle velocities and $\mathbf{a}_{\text{ext},x} \in \mathbb{R}^{n_p}$ is a vector of external acceleration in the x -direction independent of time and position. For the y -component it suffices to substitute the corresponding velocity and difference matrices $\mathbf{v}, Y^d, V^d$, as well as the corresponding external force $\mathbf{a}_{\text{ext},y}$.

2.3.3 Continuity equation

Rewriting the continuity equation (2.23) by applying the dot product yields in two dimensions

$$\frac{d\rho_i}{dt} = \sum_j m_j (X_{ij}^d U_{ij}^d + Y_{ij}^d V_{ij}^d) S_{ij}, \quad (2.43)$$

and similarly for a three dimensional flow. Making use of the Hadamard product once again yields

$$\frac{d\boldsymbol{\rho}}{dt} = [(X^d \odot U^d + Y^d \odot V^d) \odot S] \mathbf{m}. \quad (2.44)$$

2.3.4 Full model

The full model is given by the matrix momentum and continuity equations (2.42) and (2.44),

$$\begin{aligned}\frac{d\mathbf{u}}{dt} &= [(-c_s^2 X^d \odot \tilde{\rho} \odot (\rho \rho^T) + 4\nu U^d \odot \hat{\rho}) \odot S] \mathbf{m} + \mathbf{a}_{\text{ext},x}, \\ \hat{\rho} &:= \rho \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} \rho^T, \\ \tilde{\rho} &:= \rho \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} \rho^T - 2\rho_0 \mathbf{1}_{n_p \times n_p}, \\ \frac{d\rho}{dt} &= [(X^d \odot U^d + Y^d \odot V^d) \odot S] \mathbf{m}.\end{aligned}\tag{2.45}$$

With also an equation for $\mathbf{v}$ the particle velocity in the y -direction. Hence, this system of ODEs has

$$n_p(n_d + 1)\tag{2.46}$$

degrees of freedom (dofs), where n_p is the number of particles and n_d the number of spatial dimensions considered. Since n_p can be very large, we aim to reduce the number of dofs.

2.3.5 Weakly compressible assumption

Before deriving a reduced order model for (2.45), we make an extra assumption on the density, to simplify this model. As mentioned above in Section 2.2.2, as there is a nonzero change in density we are in a weakly compressible case. One key aspect here is that we can enforce the speed of sound artificially so that variations in density are of the order $\sim 1\%$ [1]. This fact allows us to assume, in the denominator of the force terms only, that for all particles i , $\rho_i \approx \rho_0$. Note that this cannot hold in the equation of state, as we would then have constant zero pressure. Under this assumption,

$$\frac{d\mathbf{u}}{dt} = [(-c_s^2 X^d \odot (\rho \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} \rho^T - 2\rho_0 \mathbf{1}_{n_p \times n_p}) + 4\nu U^d) \odot S] \mathbf{m} + \mathbf{a}_{\text{ext},x},\tag{2.47}$$

$$\frac{d\rho}{dt} = [(X^d \odot U^d + Y^d \odot V^d) \odot S] \mathbf{m},\tag{2.48}$$

which is the full order model for which we aim to reduce the dofs.

2.3.6 Boundary conditions

For the boundary conditions in matrix form, we may assume that the position of the particles, given by $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{n_p \times n_d}$ at some time t is in fact written as

$$\mathbf{x}(t) = \mathbf{x}_0 + \mathbf{x}'(t),\tag{2.49}$$

and similarly for $\mathbf{y}$, where $\mathbf{x}_0$ is the initial position of all particles – including the boundary particles. That is, we assume that $\mathbf{x}'_i(t) = 0$, for $i \in B$ a boundary particle, for all times, since the boundary particles can be assumed to not move over time. Hence, only $\mathbf{x}_0$ contains information about the position in x -direction of the boundary particles. By doing so, we consider similar boundary conditions to Domingues *et al.* [6], as detailed in Section 2.2.5.

3 Reduced Order Model

The full model as given in (2.47)-(2.48) has $n_p(n_d + 1)$ dofs. We aim to reduce this model, by looking at Singular Value Decomposition (SVD) and Proper Orthogonal Decomposition (POD). We then apply Galerkin projection, to obtain a canonical Linear Time-Invariant (LTI) system.

3.1 SVD/POD

3.1.1 Definitions

A Singular Value Decomposition (SVD) of $A \in \mathbb{R}^{n \times m}$ is a decomposition

$$A = \Phi \Sigma \Psi^T, \quad (3.1)$$

where $\Phi \in \mathbb{R}^{n \times n}$, $\Psi \in \mathbb{R}^{m \times m}$ are orthogonal and $\Sigma \in \mathbb{R}^{n \times m}$ is a matrix of decreasing singular values. We can obtain a low-rank approximation by retaining only r many singular values:

$$A \approx \Phi_r \Sigma_r \Psi_r^T, \quad (3.2)$$

where $\Phi_r \in \mathbb{R}^{n \times r}$, $\Psi_r \in \mathbb{R}^{m \times r}$ and $\Sigma_r \in \mathbb{R}^{r \times r}$. That is, only the r first columns of Φ and r first rows of Ψ^T corresponding to the r largest singular values are computed. In this paper we only consider low-rank approximations and truncated SVDs, so we drop the subscripts r for the sake of clarity.

A similar decomposition can be obtained for functions, namely $f(\mathbf{x}, t)$, with $\mathbf{x} \in \mathbb{R}^{n_d}$, $t \in \mathbb{R}$, can be decomposed and approximated by [2, 4]

$$f(\mathbf{x}, t) = \sum_{k=1}^{\infty} \Phi_k(\mathbf{x}) q_k(t) \approx \sum_{k=1}^r \Phi_k(\mathbf{x}) q_k(t), \quad (3.3)$$

where Φ_k is a set of orthonormal basis functions. This is referred to as a Proper Orthogonal Decomposition (POD). By comparing this with the above truncated SVD for matrices (3.2), we see that we can obtain a POD for matrices $A \in \mathbb{R}^{n_p \times n_t}$, n_p being the number of particles and n_t the number of timesteps of a SPH simulation,

$$A \approx \Phi q, \quad (3.4)$$

where $\Phi \in \mathbb{R}^{n_p \times n_r}$ and $q = \Sigma \Phi^T \in \mathbb{R}^{n_r \times n_t}$. If A is a matrix which depends on both space and time, then by the POD it can be assumed that Φ contains the spatial modes, while q the temporal components [4]. In the Lagrangian framework, the spatial dependence of Φ is not to a fixed position in space, but rather to a fixed particle i .

3.1.2 Decomposing the variables

Suppose we run one simulation of SPH, for some number of particles n_p and some number of timesteps n_t , where the timesteps are small enough to have a stable solution, but perhaps we run only for a few seconds instead of the full timescale we wish to simulate. From this, we obtain

snapshot matrices, containing the information of all the particles at all timesteps. Looking first at the position, in the case of a two dimensional flow, we have position matrices in the x - and y -directions respectively: $X, Y \in \mathbb{R}^{n_p \times n_t}$. Applying POD as given in (3.4) yields the decomposition of the position matrices as

$$X \approx \Phi_x q_x, \quad Y \approx \Phi_y q_y, \quad (3.5)$$

where $\Phi_x \in \mathbb{R}^{n_p \times r_x}$, $q_x \in \mathbb{R}^{r_x \times n_t}$ and $\Phi_y \in \mathbb{R}^{n_p \times r_y}$, $q_y \in \mathbb{R}^{r_y \times n_t}$, for r_x, r_y the respective truncation orders. The velocity is given by the time derivative of position, and since only q contains temporal information, we obtain the respective velocity matrices $U, V \in \mathbb{R}^{n_p \times n_t}$ as

$$U = \dot{X} \approx \Phi_x \dot{q}_x, \quad V = \dot{Y} \approx \Phi_y \dot{q}_y. \quad (3.6)$$

The position and velocity at a fixed timestep t_k is found by

$$\mathbf{x}(t_k) := \Phi_x q_x(t_k) \quad \mathbf{u}(t_k) := \Phi_x \dot{q}_x(t_k), \quad (3.7)$$

where $q_x(t_k) \in \mathbb{R}^{r_x}$ is the k -th column of q_x . The same result holds for the position and velocity in y -direction. Proceeding similarly for the difference in position in x -direction between particles i and j at timestep t_k , defined by $X_{ij}^d := \mathbf{x}_i - \mathbf{x}_j$, we have:

$$\begin{aligned} X_{ij}^d &\approx \Phi_{x_i} q_x(t_k) - \Phi_{x_j} q_x(t_k) \\ &= (\Phi_{x_i} - \Phi_{x_j}) q_x(t_k) \\ &=: \Phi_{x_{ij}}^d q_x(t_k), \end{aligned} \quad (3.8)$$

where $\Phi_{x_i} \in \mathbb{R}^{1 \times r_x}$ is the i -th row of Φ_x , and we again make use of the shorthand notation with the superscript d to denote a difference. We now have the ij -th entry of the matrix X^d , where each entry is multiplied by $q_x(t_k)$. To factor out this term, we make use of the *Kronecker product* ($\otimes$)

$$\begin{aligned} X^d &\approx \Phi_x^d (I_{n_p} \otimes q_x(t_k)), \\ \implies U^d &\approx \Phi_x^d (I_{n_p} \otimes \dot{q}_x(t_k)), \end{aligned} \quad (3.9)$$

where we obtain the difference in velocity in the x -direction by taking a time derivative, and the identity matrix $I_{n_p} \in \mathbb{R}^{n_p \times n_p}$ is used for the correct factorisation of the temporal component.

Note that Φ_x^d can be seen both as a matrix of size $n_p \times n_p r_x$ and a tensor of size $n_p \times n_p \times r_x$. We can illustrate in the case for $n_p = 3$, $r_x = 2$, denoting

$$\Phi_{x_i} = [\phi_i^1 \quad \phi_i^2] \in \mathbb{R}^{1 \times 2}. \quad (3.10)$$

The difference matrix is then given by:

$$\begin{aligned}
\Phi_x^d &= \begin{bmatrix} \Phi_{x_1} - \Phi_{x_1} & \Phi_{x_1} - \Phi_{x_2} & \Phi_{x_1} - \Phi_{x_3} \\ \Phi_{x_2} - \Phi_{x_1} & \Phi_{x_2} - \Phi_{x_2} & \Phi_{x_2} - \Phi_{x_3} \\ \Phi_{x_3} - \Phi_{x_1} & \Phi_{x_3} - \Phi_{x_2} & \Phi_{x_3} - \Phi_{x_3} \end{bmatrix} \\
&= \begin{bmatrix} (0, 0) & (\phi_1^1 - \phi_2^2, \phi_1^2 - \phi_2^2) & (\phi_1^1 - \phi_3^1, \phi_1^2 - \phi_3^2) \\ (\phi_2^1 - \phi_1^1, \phi_2^2 - \phi_1^2) & (0, 0) & (\phi_2^1 - \phi_3^1, \phi_2^2 - \phi_3^2) \\ (\phi_3^1 - \phi_1^1, \phi_3^2 - \phi_1^2) & (\phi_3^1 - \phi_2^1, \phi_3^2 - \phi_2^2) & (0, 0) \end{bmatrix} \in \mathbb{R}^{n_p \times n_p \times r_x} \quad (3.11) \\
&= \begin{bmatrix} 0 & 0 & \phi_1^1 - \phi_2^2 & \phi_1^2 - \phi_2^2 & \phi_1^1 - \phi_3^1 & \phi_1^2 - \phi_3^2 \\ \phi_2^1 - \phi_1^1 & \phi_2^2 - \phi_1^2 & 0 & 0 & \phi_2^1 - \phi_3^1 & \phi_2^2 - \phi_3^2 \\ \phi_3^1 - \phi_1^1 & \phi_3^2 - \phi_1^2 & \phi_3^1 - \phi_2^1 & \phi_3^2 - \phi_2^2 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{n_p \times n_p \times r_x}.
\end{aligned}$$

Hence, when considering (3.9), Φ_x^d must be of size $n_p \times n_p \times r_x$ for the dimensions to match in the matrix-multiplication.

Finally, we want to decompose the density. Consider the snapshot matrix $P \in \mathbb{R}^{n_p \times n_t}$. Then, applying POD yields

$$P \approx \Phi_\rho q_\rho, \quad (3.12)$$

where $\Phi_\rho \in \mathbb{R}^{n_p \times r_\rho}$ and $q_\rho \in \mathbb{R}^{r_\rho \times n_t}$. At timestep t_k the density vector is

$$\rho(t_k) \approx \Phi_\rho q_\rho(t_k), \quad (3.13)$$

where $q_\rho(t_k) \in \mathbb{R}^{r_\rho}$ is the k -th column of q_ρ .

3.1.3 Decomposing the model

Recalling the full-order model (2.47)-(2.48), we can substitute the decompositions of the variables given in the previous section, to obtain:

$$\begin{aligned}
\Phi_x \ddot{q}_x(t) &= \left\{ \left[-c_s^2 \Phi_x^d (I_{n_p} \otimes q_x(t)) \odot (\Phi_\rho q_\rho(t) \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} q_\rho(t)^T \Phi_\rho^T - 2\rho_0 \mathbf{1}_{n_p \times n_p}) \right. \right. \\
&\quad \left. \left. + 4\nu \Phi_x^d (I_{n_p} \otimes \dot{q}_x(t)) \right] \odot S \right\} \mathbf{m} + \Phi_x^T \mathbf{a}_{\text{ext}, x}, \quad (3.14)
\end{aligned}$$

$$\begin{aligned}
\Phi_\rho \dot{q}_\rho(t) &= \left\{ \left[\Phi_x^d (I_{n_p} \otimes q_x(t)) \odot \Phi_x^d (I_{n_p} \otimes \dot{q}_x(t)) \right. \right. \\
&\quad \left. \left. + \Phi_y^d (I_{n_p} \otimes q_y(t)) \odot \Phi_y^d (I_{n_p} \otimes \dot{q}_y(t)) \right] \odot S \right\} \mathbf{m}, \quad (3.15)
\end{aligned}$$

where only $q_x(t)$, $\dot{q}_x(t)$, $\ddot{q}_x(t)$, $q_\rho(t)$, $\dot{q}_\rho(t)$ and S depend on time, and all other terms are constant and can be precomputed. Recall that each of those aforementioned temporal terms have dimensions r_x and r_ρ – and r_y for the y -direction. Typically, these truncation orders are much smaller than the full order n_p , hence we have effectively reduced the number of degrees of freedom.

3.2 Galerkin projection

The reduced model (3.14)-(3.15) does not give explicit equations for the evolution of the temporal components q , but rather for Φq . To get rid of the Φ 's on the left hand side we recall that they come from singular values decompositions, and hence they are orthogonal, that is: $\Phi^T \Phi = I$. Therefore, pre-multiplying both equations by the corresponding transpose yields:

$$\ddot{q}_x(t) = \Phi_x^T \left\{ \left[-c_s^2 \Phi_x^d(I_{n_p} \otimes q_x(t)) \odot (\Phi_\rho q_\rho(t) \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} q_\rho(t)^T \Phi_\rho^T - 2\rho_0 \mathbf{1}_{n_p \times n_p}) \right. \right. \\ \left. \left. + 4\nu \Phi_x^d(I_{n_p} \otimes \dot{q}_x(t)) \right] \odot S \right\} \mathbf{m} + \Phi_x^T \mathbf{a}_{\text{ext},x}, \quad (3.16)$$

$$\dot{q}_\rho(t) = \Phi_\rho^T \left\{ \left[\Phi_x^d(I_{n_p} \otimes q_x(t)) \odot \Phi_x^d(I_{n_p} \otimes \dot{q}_x(t)) \right. \right. \\ \left. \left. + \Phi_y^d(I_{n_p} \otimes q_y(t)) \odot \Phi_y^d(I_{n_p} \otimes \dot{q}_y(t)) \right] \odot S \right\} \mathbf{m}. \quad (3.17)$$

We have then obtained a reduced order model, with degrees of freedom:

$$r_x + r_y + r_\rho \ll n_p(n_d + 1). \quad (3.18)$$

However, we must note the presence of the radial derivative S in this reduced order model. This term depends *nonlinearly* on the *full* representation of the position X, Y , and therefore, despite indeed having a model with lower order, the presence of this radial derivative term adds some complexity that we aim to diminish.

3.3 Boundary conditions

We proceed in a similar way as in Section 2.3.6 for the boundary conditions in matrix form. That is, we split the POD of the position (3.5) to include the initial position

$$X \approx \Phi_x q_x + X_0, \quad (3.19)$$

and hence for the difference in position in the x -direction we now obtain, similarly to (3.9),

$$X^d \approx \Phi_x^d(I_{n_p} \otimes q_x(t)) + X_0^d, \quad (3.20)$$

where $X_0^d \in \mathbb{R}^{n_p \times n_p}$ is the initial difference in position in x -direction between particles i and j . Implementing (3.20) into (3.16)-(3.17) gives a ROM which admits the desired boundary conditions:

$$\ddot{q}_x(t) = \Phi_x^T \left\{ \left[-c_s^2 \{ \Phi_x^d(I_{n_p} \otimes q_x(t)) + X_0^d \} \odot (\Phi_\rho q_\rho(t) \mathbf{1}_{1 \times n_p} + \mathbf{1}_{n_p \times 1} q_\rho(t)^T \Phi_\rho^T - 2\rho_0 \mathbf{1}_{n_p \times n_p}) \right. \right. \\ \left. \left. + 4\nu \Phi_x^d(I_{n_p} \otimes \dot{q}_x(t)) \right] \odot S \right\} \mathbf{m} + \Phi_x^T \mathbf{a}_{\text{ext},x}, \quad (3.21)$$

$$\dot{q}_\rho(t) = \Phi_\rho^T \left\{ \left[\{ \Phi_x^d(I_{n_p} \otimes q_x(t)) + X_0^d \} \odot \Phi_x^d(I_{n_p} \otimes \dot{q}_x(t)) \right. \right. \\ \left. \left. + \{ \Phi_y^d(I_{n_p} \otimes q_y(t)) + Y_0^d \} \odot \Phi_y^d(I_{n_p} \otimes \dot{q}_y(t)) \right] \odot S \right\} \mathbf{m}. \quad (3.22)$$

4 Radial derivative

The radial derivative S of the kernel W , as defined in (2.9), is represented in the reduced order model (3.21)-(3.22) by a matrix of dimension $n_p \times n_p$ where the ij -th entry is the radial derivative of W_{ij} with respect to particle i . Typically, kernels depend on the distance between particles, and hence the radial derivative depends nonlinearly on the position. Furthermore, it also depends on the full representation of position so we aim to rewrite it in terms of the truncated temporal modes.

When dealing with nonlinear terms, there are two ‘simple’ solutions that may come to mind: freezing and linearising. Freezing consists in only updating the term once every few timesteps, and keeping it constant between updates [7]. However, here, the radial derivative may change significantly between two timesteps, as it depends on the position between particles, which itself changes at every timestep. Particularly so in the case of sloshing, where particles are not bound to one another. Therefore, freezing does not appear to be a suitable choice for such application.

Consequently, we first attempt linearisation, in the specific case of a Gaussian kernel. Then, we look at Random Fourier Features (RFF), and finally we turn to interpolation – both illustrated with the Wendland kernel.

4.1 Linearisation

In this section, we aim to linearise the radial derivative in the particular case of a Gaussian kernel. We choose this kernel as it is rather simple and provides worthy insight into this potential solution.

We denote the distance between particles i and j by $r_{ij} = \|(\mathbf{x}_i - \mathbf{x}_j, \mathbf{y}_i - \mathbf{y}_j)\|$, where $(\mathbf{x}_i, \mathbf{y}_i)$ is the position of particle i at some time t . Considering the Gaussian kernel and its radial derivative as given in (2.11) and (2.12) respectively, we have:

$$W_{ij} = \alpha_d \exp\left(-\frac{r_{ij}^2}{\varepsilon^2}\right), \quad S_{ij} = \frac{2\alpha_d}{\varepsilon^2} \exp\left(-\frac{r_{ij}^2}{\varepsilon^2}\right) = \frac{2}{\varepsilon^2} W_{ij}, \quad (4.1)$$

where α_d is a normalisation constant.

4.1.1 Distance matrix

The Gaussian kernel W and its radial derivative S both depend on the distance between particles. Hence, it is of interest to define a matrix for which the ij -th entry is this distance. We have:

$$\begin{aligned} D_{ij}^2 &:= \|(X_{ij}^d, Y_{ij}^d)\|_2^2 = (X_{ij}^d)^2 + (Y_{ij}^d)^2 \\ &= \left(\Phi_{x_{ij}}^d q_x(t)\right)^2 + \left(\Phi_{y_{ij}}^d q_y(t)\right)^2 \\ &= q_x(t)^T (\Phi_{x_{ij}}^d)^T (\Phi_{x_{ij}}^d) q_x(t) + q_y(t)^T (\Phi_{y_{ij}}^d)^T (\Phi_{y_{ij}}^d) q_y(t). \end{aligned} \quad (4.2)$$

Using of the *Hadamard power* and the Kronecker product we obtain the distance matrix squared:

$$D^{\circ 2} = (I_{n_p} \otimes q_x(t))^T (\Phi_x^d)^T (\Phi_x^d) (I_{n_p} \otimes q_x(t)) + (I_{n_p} \otimes q_y(t))^T (\Phi_y^d)^T (\Phi_y^d) (I_{n_p} \otimes q_y(t)), \quad (4.3)$$

where in the case of a three dimensional flow we simply add the corresponding third term.

4.1.2 Fréchet derivative and linearisation

The Fréchet derivative of the matrix exponential $F(X) = e^X$ at A in the direction E is [19]:

$$D_F(A, E) = \int_0^1 e^{A(1-s)} E e^{As} ds. \quad (4.4)$$

The Taylor expansion for function of matrices is given by [5]

$$F(A + E) = \sum_{k=0}^{\infty} \frac{1}{k!} D_F^{[k]}(A, E), \quad (4.5)$$

where $D_F^{[k]}(A, E)$ is the k -th order Fréchet derivative of F at A in the direction of E . Taking only the first two terms of the sum to obtain a linearisation of the matrix exponential yields:

$$e^{A+E} \approx e^A + \int_0^1 e^{A(1-s)} E e^{As} ds. \quad (4.6)$$

Taking $E = -D^{\circ 2}/\varepsilon^2$, where $D^{\circ 2}$ is the distance matrix squared as in (4.3), and $A = 0$, we can linearise the matrix exponential of the distance as

$$\exp\left(-\frac{D^{\circ 2}}{\varepsilon}\right) \approx I_{n_p} - \frac{1}{\varepsilon^2} D^{\circ 2}. \quad (4.7)$$

Hence, linearising the radial derivative of a Gaussian kernel yields

$$S \approx \frac{2\alpha_d}{\varepsilon^2} \left(I_{n_p} - \frac{1}{\varepsilon^2} D^{\circ 2} \right) =: \tilde{S}. \quad (4.8)$$

We can then note that as the distance between particles increases, this linearisation becomes less accurate. Furthermore, in the case of the Gaussian kernel, we have that as the distance between particles increases, the kernel and its radial derivative both tend to zero. That is not the case anymore for the linearisation, where as the distance increases, $\tilde{S}$ tends to $-\infty$ in the limit. This would mean that when considering a reconstruction with $\tilde{S}$ as in (4.8), particles far from one another still influence each other to a nonnegligible extent – which is a unphysical behaviour.

Moreover, each choice of kernel would yield a different linearisation. A Fréchet derivative must be computed in any case, which may not always be trivial. Hence, even if linearisation seems to be an obvious first choice when dealing with a nonlinear term, in our case of reduced order model it does not bring the desired results. This is why we turn to other alternatives, such as Random Fourier Features and SVD interpolation.

4.2 Random Fourier Features

Another approach to dealing with the nonlinear radial derivative term is via Random Fourier Features (RFF). The underlying fact here is that any positive definite function $k(\mathbf{x}, \mathbf{y})$, $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{n_d}$, defines an inner product and a lifting ϕ so that

$$k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle, \quad (4.9)$$

where ϕ is called a *reproducing kernel* [15]. Rahimi and Recht [17] propose instead an approximation via a *randomized feature map* $\mathbf{z} : \mathbb{R}^{n_d} \rightarrow \mathbb{R}^N$, where $\mathbb{R}^N$ has lower dimension than $\mathbb{R}^{n_d}$, such that

$$k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle \approx \mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{y}). \quad (4.10)$$

This then yields a low-dimensional approximation. A key element is Bochner's theorem [9, 18], which states that a continuous function of the form

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{x} - \mathbf{y}) \quad (4.11)$$

is positive definite if, and only if, $k(\delta)$ is the Fourier transform of a nonnegative measure. Hence, when considering the kernel $W_{ij} = W(\mathbf{x}_i - \mathbf{x}_j, \epsilon)$, its Fourier transform $p(\boldsymbol{\omega})$ is a probability density function [9]. We then introduce the Random Fourier Features [17]:

$$\mathbf{z}(\mathbf{x}) = [z_{\boldsymbol{\omega}_1}(\mathbf{x}) \quad \dots \quad z_{\boldsymbol{\omega}_N}(\mathbf{x})], \quad z_{\boldsymbol{\omega}_m}(\mathbf{x}) = \sqrt{2} \cos(\boldsymbol{\omega}_m^T \mathbf{x} + b_m), \quad (4.12)$$

where $\boldsymbol{\omega}_w$ is drawn from the Fourier transform $p(\boldsymbol{\omega})$ and b_m is drawn uniformly on $[0, 2\pi]$. These Fourier features approximate the kernel as

$$W(\mathbf{x}_i - \mathbf{x}_j) \approx \mathbf{z}(\mathbf{x}_i)^T \mathbf{z}(\mathbf{x}_j), \quad (4.13)$$

where $\mathbf{x}_i \in \mathbb{R}^{n_d}$ is the position of particle i at some time t . Making use of the splitting of the spatial dimensions, as well as the application of POD as in Section 3.1.2 we get, for $n_d = 2$,

$$\mathbf{x}_i = [\mathbf{x}_i \quad \mathbf{y}_i] = [\Phi_{x_i} q_x(t) \quad \Phi_{y_i} q_y(t)] \in \mathbb{R}^2, \quad (4.14)$$

where $\Phi_{x_i} \in \mathbb{R}^{1 \times n_r}$ is the i -th row of Φ_x and $q_x(t) \in \mathbb{R}^{n_r}$. Note that here we aim to approximate the radial derivative of the kernel, hence we also need to take the derivative of the Fourier features (4.12).

The reconstruction of the radial derivative of the Wendland kernel (2.14) for $N = 50$ and 500 features is given in Figure 1. We can see that for 50 features, there are some large wiggles appearing at the tail end of the reconstruction. These wiggles are a problem, since the Wendland kernel – and hence its radial derivative – is set to zero for particles with a distance larger than some threshold.

However, in the reconstruction, particles which are far from each other would still influence one another, which is non-physical. Increasing the number of features to 500 smoothens the wiggles, but the noise does not entirely disappear. We would need to increase the number of features even more

to have the noise be negligible, but in doing so we do not have a reduced order model anymore, defeating the purpose of using RFF in the first place.

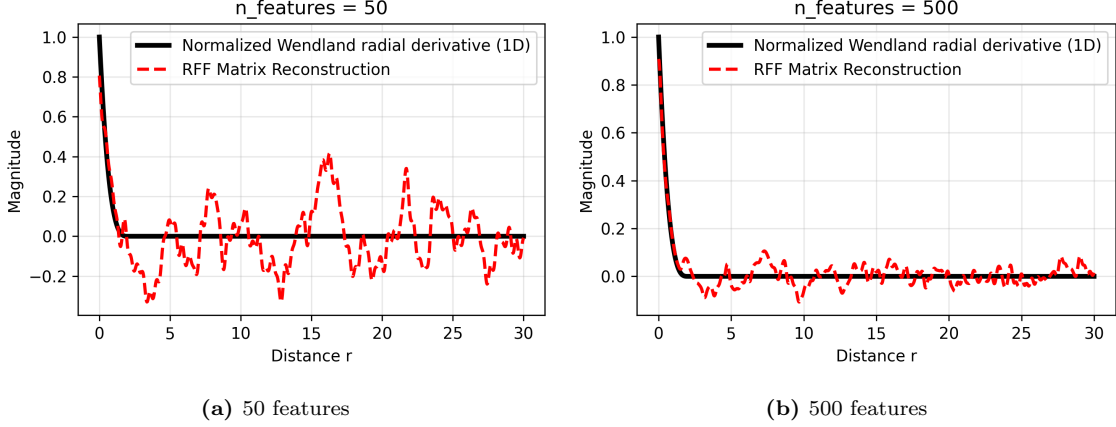


Figure 1. Reconstruction of the radial derivative of the Wendland kernel (2.14) for 50 and 500 Fourier features.

4.3 SVD Interpolation

Since neither linearisation nor random Fourier features give a desired result for the reconstruction of the radial derivative of the smoothing kernel, we turn to interpolation. The idea here is to run one simulation of SPH – for a relatively small time – to obtain snapshot matrices for the position X, Y as well as for the radial derivative S of the kernel of choice W , for all particles and all timesteps. Then, we apply POD to the snapshot matrices, as in (3.5), and:

$$S \approx \Phi_s q_s, \quad (4.15)$$

where the respective reduced orders r_x, r_y, r_s are chosen based on energy levels. We interpolate the temporal components, to obtain a function f which approximates

$$q_s \approx f(q_x, q_y). \quad (4.16)$$

We use here linear radial basis functions, of the form

$$f(q) = \sum_i w_i \|q - c_i\| + P(q), \quad (4.17)$$

where w_i and c_i are weights and centres of the radial basis function, and $P(q)$ is a polynomial function of q . We consider q to be a vertically stacked matrix of q_x and q_y .

In the following, we consider the radial derivative of the Wendland kernel (2.14). We run a SPH simulation for 498 particles and 996 timesteps. Figure 2 represents the normalised cumulative

energy per number of modes for the two position matrices as well as the radial derivative S . We pick truncation orders such that 90% of the energy is retained for position X, Y , and 75% for S . This yields an interpolation with only 4 modes.

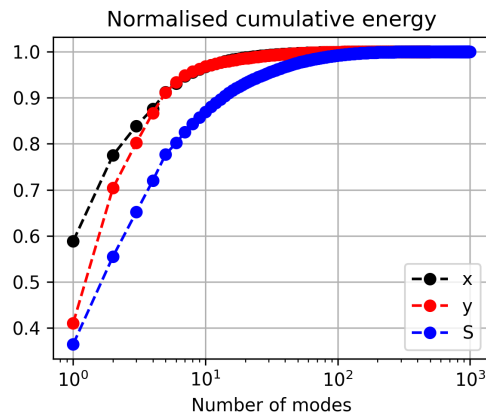


Figure 2. Normalised cumulative energy per number of modes. X, Y are position matrices, S is the normalised radial derivative of the Wendland kernel.

In order to properly analyse the result of the interpolation, we compare with the reconstruction from a simple truncation. That is, we compare the reconstruction

$$S \approx \Phi_s f(q_x, q_y), \quad (4.18)$$

with the reconstruction from a truncated SVD with the same number of modes retained – in our case 4 modes. This is illustrated in Figure 3, at timestep $t = 995$.

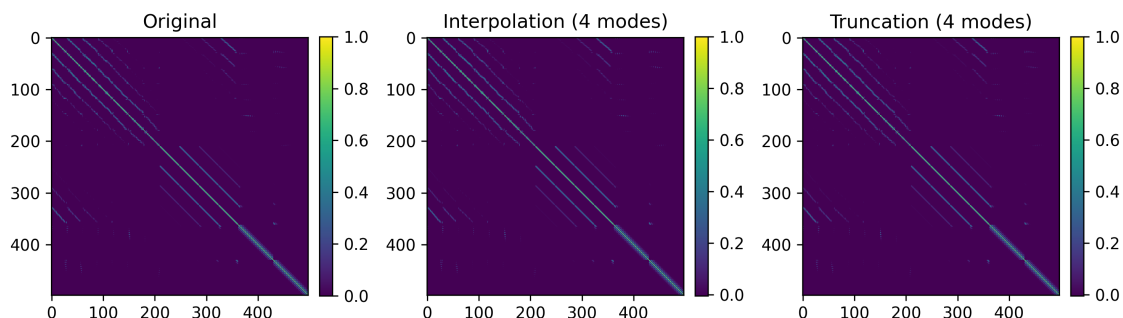


Figure 3. Normalised radial derivative of the Wendland kernel (original, left plot) with two reconstructions (interpolation, middle and truncation, right) each obtained with 4 modes, at timestep $t = 995$.

The first reconstruction (middle) is via interpolation (4.16), retaining only 4 modes for both q_x

and q_y . The second reconstruction (right) is obtained by only keeping 4 modes in q_s . In both cases we reconstruct S as $\Phi_s q_s$.

From these plots we can see that the reconstruction is similar to the original matrix, and in fact when computing the respective normalised errors in Frobenius norm, we find:

$$\frac{\|S - S_{\text{interp}}\|_F}{\|S\|_F} = 0.0905, \quad \frac{\|S - S_{\text{trunc}}\|_F}{\|S\|_F} = 0.0477. \quad (4.19)$$

The error for the interpolation is hence about twice as big as the error for the truncation, while still being rather small – about 10%. Furthermore, in this reconstruction, we only keep about 4 modes out of the original 498, i.e. less than 1%. We have effectively obtained an approximation of the radial derivative S in terms of the reduced temporal coefficients q_x, q_y , which is accurate and drastically reduces the number of modes needed for the computation.

5 Conclusion and discussion

This paper focuses on a Lagrangian reduced order model (ROM) for applications in sloshing and free-surface flows. The Lagrangian model stems from interpolating the equations of motion with a smoothing kernel, then applying a discrete approximation to obtain the smoothed-particle hydrodynamics (SPH) equations. These equations are written as sums over neighbouring particles. The first step of obtaining the reduced order model is rewriting these sums as matrix-vector operations. From this, truncated singular value decomposition is applied, together with Galerkin projection, yielding the desired ROM.

However, this model contains a nonlinear term, corresponding to the radial derivative of the smoothing kernel. We analyse three possible ways of dealing with this term: linearisation, random Fourier features (RFF), and interpolation via truncated singular value decomposition. The former two do not produce suitable results. Namely, they both introduce influence from non-neighbouring particles, which is unphysical, and thus could severely affect the prediction capabilities of the ROM. In fact, it is expected that the influence of particle j on particle i decreases as the distance increases, and is nonexistent when the distance is larger than some fixed radius. Hence, neither linearisation nor RFF are adequate method to treat this radial derivative.

On the other hand, interpolation shows a favourable outcome. Indeed, we are able to reconstruct the radial derivative with up to 10% error, while only using less than 1% of the modes. This allows us to significantly reduce the dimensionality of the problem, as well as the number of degrees of freedom of the model.

Nonetheless, from this reconstruction, we cannot quite tell whether far away particles influence each other or not. Therefore, one must thread carefully when implementing this interpolation in the ROM, as some errors may still be carried along.

The next step to take in the research on reduced order Lagrangian models would be to implement the model given here in an ODE solver. The idea would be to run one SPH simulation, for a certain number of particles for a limited time – i.e. not a full simulation. Then, one could reconstruct the

ROM from the resulting snapshot matrices, and use this model for e.g. optimisation or longer time simulations.

Such implementations have not been performed here due to the short duration of the project.

Based on the results of the reduced order model simulations, some further research could be done on the assumptions that were made in this paper. In particular, we make use of the weak compressibility assumption, allowing us to replace the density of the particles in the force term by the base density. This may be too strong of an assumption, and in that case, one could look at deriving a ROM without it, that is, directly from the full matrix model as given in (2.45).

Moreover, some other method to treat the nonlinear radial derivative terms could be employed. When considering the interpolation, we only look at the radial derivative itself, and not how it fits within the model. Other methods – such as approximation by Gaussian functions – may yield more advantageous results.

Acknowledgements

I would like to thank Samuel Ahizi for his precious help during my time at VKI, and my parents for their continuous support and encouragement in my studies. I also extend my gratitude towards my landlord, Claire, as well as all the incredible people I have met in Belgium and who made this time fun and enjoyable.

6 References

- [1] M. Becker and T. Matthias. “Weakly compressible SPH for free surface flows”. *Proceedings of the 2007 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 2007.
- [2] F. Chinesta, R. Keunings, and A. Leygue. *The Proper Generalized Decomposition for Advanced Numerical Simulations*. Springer, 2013.
- [3] S. J. Cummins and M. Rudman. “An SPH Projection Method”. *Journal of Computational Physics* (1999).
- [4] S. Dawson. “The Proper Orthogonal Decomposition”. *Data-Driven Fluid Mechanics: Combining First Principles and Machine Learning*. Cambridge University Press, 2023.
- [5] E. Deadman and S. Relton. “Taylor’s Theorem for Matrix Functions with Applications to Condition Number Estimation”. *Manchester Institute for Mathematical Sciences School of Mathematics* (2015).
- [6] J. M. Domínguez et al. “State-of-the-art SPH solver DualSPHysics: from fluid dynamics to multiphysics problems”. *Computational Particle Mechanics* (2021).
- [7] L. Fang et al. “Proper Orthogonal Decomposition-based Model-Order Reduction for Smoothed Particle Hydrodynamics Simulation”. *arXiv preprint arXiv:2507.19825* (2025).
- [8] R. A. Gingold and J. J. Monaghan. “Smoothed particle hydrodynamics: theory and application to non-spherical stars”. *Monthly Notices of the Royal Astronomical Society* (1977).

- [9] A. Hinrichs and J. Vybírál. “On positive positive-definite functions and Bochner’s Theorem”. *Journal of Complexity* (2011).
- [10] G. R. Liu and M. B. Liu. “Coupling SPH with Molecular Dynamics for Multiple Scale Simulations”. *Smoothed Particle Hydrodynamics*. World Scientific, 2003.
- [11] E. Y. M. Lo and S. Songdong. “Simulation of near-shore solitary wave mechanics by an incompressible SPH method”. *Applied Ocean Research* (2002).
- [12] L. B. Lucy. “A numerical approach to the testing of the fission hypothesis”. *Astronomical Journal* (1977).
- [13] J. J. Monaghan. “Simulating Free Surface Flows with SPH”. *Journal of Computational Physics* (1994).
- [14] M. Müller, D. Charypar, and M. Gross. “Particle-based fluid simulation for interactive applications”. *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 2003.
- [15] V. I. Paulsen and M. Raghupathi. *An Introduction to the Theory of Reproducing Kernel Hilbert Spaces*. Cambridge University Press, 2016.
- [16] J. Pinho et al. “Advancing Cryogenic Fluid Research: the von Karman Institute’s Contribution to Aerospace, Energy, and Industrial Applications”. *11th European Conference for Aeronautics and Space Sciences (EUCASS)*. 2025.
- [17] A. Rahimi and B. Recht. “Random Features for Large-Scale Kernel Machines”. *Advances in Neural Information Processing Systems* (2007).
- [18] W. Rudin. *Fourier Analysis on Groups*. Wiley Classics Library, 1990.
- [19] C. Van Loen. “The Sensitivity of the Matrix Exponential”. *SIAM Journal on Numerical Analysis* (1977).
- [20] D. Violeau. *Fluid mechanics and the SPH method: theory and applications*. Oxford University Press, 2012.
- [21] H. Wendland. “Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree”. *Advances in Computational Mathematics* (1995).